

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Karpaga shree

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

```
if (x > y || p < q)
```

Intermediate code:

```
if x > y goto L1
```

```
if p < q goto L1
```

```
goto L2
```

```
L1: ...
```

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control

Given:

Instruction Target

200: if x==y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ASSESSMENT TOOL 3 - DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Understanding of Data

The given three-address code (TAC) consists of four sequential instructions, each performing exactly one operator with at most two operands, and storing the result in a temporary variable. t1 holds the result of $x + y$, t2 holds the result of $t1 * z$, t3 holds the result of $t2 / w$, and t4 holds the result of $t3 - p$. The data shows a linear chain of dependency: each instruction consumes the temporary produced by the previous instruction, meaning the original expression being evaluated is the single nested expression $((((x + y) * z) / w) - p)$.

Analysis - Construction of the Expression Tree

Since each temporary is used exactly once as an operand of the next operation, the expression tree is built bottom-up from the innermost operation outward. The leaves of the tree are the source-level identifiers (x, y, z, w, p) and every internal node is a binary operator whose two children are the operands of that instruction. Tracing the dependency chain $t1 \rightarrow t2 \rightarrow t3 \rightarrow t4$ produces the following tree:

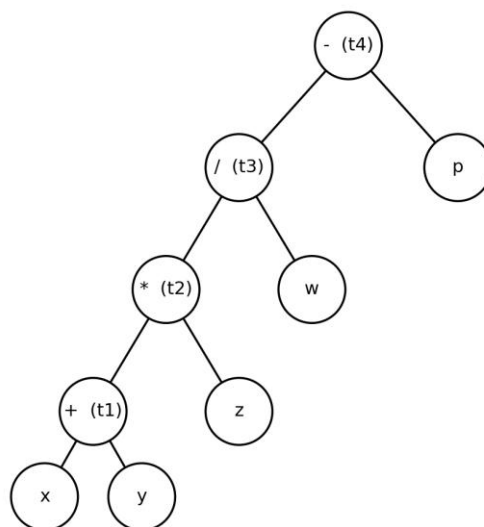


Figure 1: Expression tree equivalent to the given three-address code.

Application of Concepts

Each internal node in the tree corresponds directly to one TAC instruction, and a post-order traversal of the tree (left child, right child, then node) regenerates exactly the given instruction sequence t1, t2, t3, t4. This confirms that three-address code is essentially a linearized, flattened form of an expression tree in which every internal node is assigned a unique temporary name. The temporary variables t1-t4 therefore act as explicit handles to intermediate sub-results, which is what allows the compiler to generate code for deeply nested expressions using only simple two-operand instructions, and to feed each intermediate value directly into a register during code generation instead of re-parsing the original expression.

Conclusion

Temporary variables improve intermediate code generation in three important ways: (1) they decompose a complex nested expression into a sequence of simple, easily translatable machine-level operations, (2) they make dependencies between sub-expressions explicit, which simplifies register allocation and instruction scheduling, and (3) if any sub-expression such as t1 or t2 were needed again elsewhere in the program, the stored temporary can be reused directly instead of being recomputed, which is the foundation of optimizations such as common subexpression elimination (illustrated further in Q3). Overall, the temporary-based three-address form allows the compiler to generate efficient, minimal-recomputation code directly from the expression tree.

Q2. Symbol Table and Memory Allocation

Given symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Understanding of Data

The symbol table lists four variables with their declared data types and the number of bytes each occupies on a typical 32/64-bit system: p (char, 1 byte), q (double, 8 bytes), r (int, 4 bytes), and s (float, 4 bytes). The raw (unaligned) sum of these sizes is $1 + 8 + 4 + 4 = 17$ bytes.

Analysis - Alignment Requirement

A compiler cannot simply place these variables back-to-back in memory, because most processors require a variable to start at a memory address that is a multiple of its own size (its "alignment"). A char (1 byte) may start at any address, an int or float (4 bytes) must start at an address divisible by 4, and a double (8 bytes) must start at an address divisible by 8. Since p (1 byte) is followed by q (8 bytes) in the declaration order, the compiler must insert padding bytes after p so that q begins on an 8-byte boundary.

Application of Concepts - Memory Map

Applying the alignment rule to the declaration order (p, q, r, s) produces the following memory allocation map:

Variable	Offset (Start)	Size (Bytes)	Offset (End)
p (char)	0	1	1
padding	1	7	8
q (double)	8	8	16
r (int)	16	4	20
s (float)	20	4	24

Total memory required = 24 bytes (17 bytes of actual data + 7 bytes of alignment padding). Note that reordering the variables by descending size (q, r, s, p) removes the internal padding after p, but the overall block must still end on an 8-byte boundary (the alignment of its largest member, double), so the final allocated size still rounds up from 17 to 24 bytes. This shows the alignment overhead is unavoidable once a double is present, regardless of declaration order.

Conclusion

The compiler allocates 24 bytes in total for these four variables, even though their combined declared sizes are only 17 bytes. The extra 7 bytes are alignment padding required so that every variable begins at an address consistent with its own size, particularly the 8-byte-aligned double q. This demonstrates that memory organization in a compiler depends not only on data type sizes but also on hardware alignment constraints, and that careful ordering of declarations (largest data type first) can reduce internal fragmentation, although here the trailing padding to the largest alignment (8 bytes) cannot be avoided.

Q3. Optimization in Intermediate Code

Given intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Understanding of Data

The intermediate code contains four instructions computing four temporaries. Reading them in order: t1 is assigned the product of m and n; t2 is assigned the sum of p and q; t3 is assigned the product of m and n; and t4 is assigned t2 minus t3.

Analysis - Step-by-Step Trace

Step	Instruction	Observation
1	$t1 = m * n$	Computes $m * n$ and stores it in t1.
2	$t2 = p + q$	Computes $p + q$ and stores it in t2 (independent of step 1).
3	$t3 = m * n$	Recomputes $m * n$, identical expression to step 1 - redundant.
4	$t4 = t2 - t3$	Uses t2 and t3, where t3 duplicates the value already in t1.

Tracing the operands shows that step 3 ($t3 = m * n$) evaluates exactly the same expression as step 1 ($t1 = m * n$), and neither m nor n is modified between these two steps. This is a common subexpression: the same computation is performed twice for no additional benefit, wasting one multiplication and one temporary variable.

Application of Concepts - Common Subexpression Elimination

Applying common subexpression elimination (CSE), the second computation of $m * n$ (t3) is removed, and every later use of t3 is replaced by the already-available value t1.

The optimized intermediate code becomes:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1
```

This reduces the instruction count from four to three, eliminates one redundant multiplication, and frees the temporary t3 that would otherwise have to be allocated a register or memory location.

Conclusion

Optimization through common subexpression elimination and redundant computation removal directly improves execution efficiency: fewer arithmetic operations are executed at run time, fewer temporary variables compete for registers, and the resulting code is both smaller and faster without changing the program's observable behaviour. This example demonstrates why compilers perform data-flow analysis over intermediate code before final code generation - identical, side-effect-free expressions computed from unmodified operands can always be safely merged into a single computation.

Q4. Control Flow and Boolean Expression Handling

Given Boolean expression:

```
if (x > y || p < q)
```

Given intermediate code:

```
if x > y goto L1
if p < q goto L1
goto L2
L1: ...
```

Understanding of Data

The intermediate representation translates the OR (||) condition into a sequence of conditional jumps rather than computing a single Boolean value first. There are two conditional-jump instructions (for $x > y$ and $p < q$), both targeting the same true label L1, followed by an unconditional jump to the false label L2.

Analysis - Control Flow Interpretation

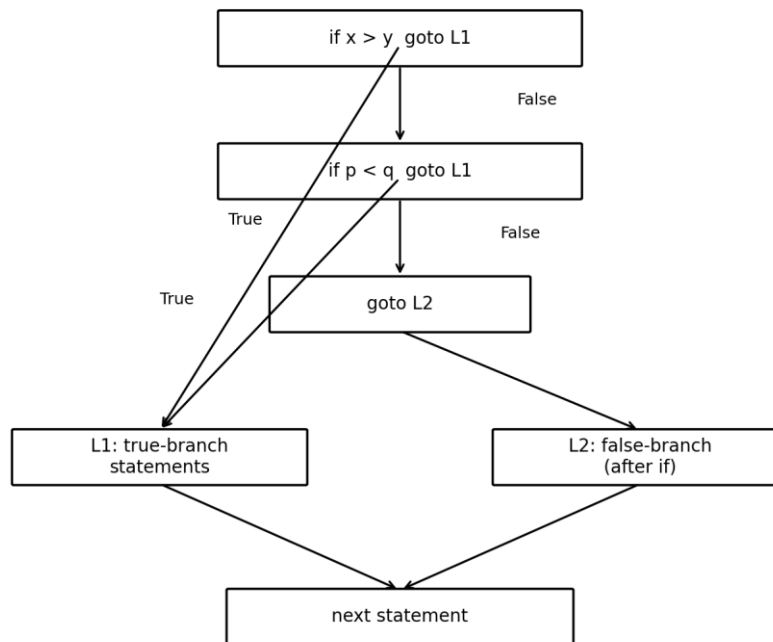


Figure 2: Control-flow translation of $\text{if } (x > y \parallel p < q)$.

Execution proceeds as follows: first, $x > y$ is evaluated. If it is true, control jumps directly to L1 (the true branch) without ever evaluating $p < q$. Only if $x > y$ is false does execution fall through to the second test, $p < q$. If that is true, control again jumps to L1; if it is also false, execution falls through to goto L2, transferring control to the false branch. This matches the semantics of logical OR: the overall condition is true as soon as any one operand is true.

Application of Concepts - Short-Circuit Evaluation

This translation implements short-circuit evaluation: the second condition ($p < q$) is only evaluated when the first condition ($x > y$) has already been found false, because a true first operand already guarantees the OR expression is true. This avoids unnecessary condition checking - and, more importantly, avoids evaluating expressions that could have side effects or be computationally expensive - whenever the first operand alone is sufficient to determine the outcome. The same jump-based translation strategy (using true/false labels instead of building a temporary Boolean value) generalizes directly to AND ($\&\&$) expressions and to nested Boolean expressions.

Conclusion

Interpreting the given intermediate representation shows that Boolean expressions are compiled into control flow (conditional jumps to true/false labels) rather than into value computation. Short-circuit evaluation minimizes the number of condition checks actually

executed at run time, improves performance for compound conditions, and correctly preserves the logical semantics of the OR operator by only proceeding to test later operands when earlier ones are insufficient to decide the result.

Q5. Backpatching and Flow Control

Given:

Instruction	Target
200: if x == y goto _	(unresolved)
201: goto _	(unresolved)

truelist = {200}

falselist = {201}

Understanding of Data

Two instructions have been generated with their jump targets left blank ("_"), because at the point of generation the compiler does not yet know the addresses of the true and false branches. Instruction 200 (a conditional jump, if x == y goto _) is recorded in truelist, meaning it must eventually jump to wherever the true-branch code begins. Instruction 201 (an unconditional jump, goto _) is recorded in falselist, meaning it must eventually jump to wherever the false-branch code begins.

Analysis - Role of Backpatching

Backpatching is the technique of generating jump instructions with incomplete (empty) targets and maintaining lists of such instructions (truelist, falselist) that all require the same eventual target address. Only when the compiler later reaches the point in the source program where the true or false branch actually starts does it know the correct target address; at that point it "patches" every instruction number stored in the corresponding list with that address, using a function typically called backpatch(list, target).

Application of Concepts - Demonstration

Suppose that, as parsing continues, the true-branch code is found to begin at address 202 (the very next instruction) and the false-branch code is found to begin at address 210. The compiler then calls backpatch(truelist, 202) and backpatch(falselist, 210), which fill in the previously empty targets:

```
200: if x == y goto 202
201: goto 210
```

Both instructions now hold complete, resolved target addresses, even though those targets were unknown at the moment the instructions were first emitted.

Conclusion

Backpatching allows a compiler to generate correct jump-based intermediate code for control constructs (if, if-else, while, for, and Boolean expressions) in a single left-to-right pass over the source program, without needing to know label addresses in advance. By deferring target-address resolution to lists (truelist, falselist) and filling them in once the branch targets become known, backpatching is essential for efficient single-pass translation of Boolean expressions and control-flow statements into intermediate code.